

Proyecto de Programación

Caja registradora de una pizzería

Python, funciones, estructuras de datos, bucles, condicionales y GitHub

Entrega individual o por parejas, según indicación del profesor.

Proyecto: Caja registradora de una pizzería

Vas a crear un programa en Python que simule la caja registradora de una pizzería. El programa deberá calcular precios de pizzas, bebidas, ofertas y pedidos, teniendo en cuenta costes de ingredientes, suministros, gastos fijos, IVA y beneficio.

El proyecto debe resolverse con los contenidos trabajados en clase: tipos simples, datos compuestos, operadores, condicionales, bucles, iteradores, funciones, Git y GitHub. No se usarán objetos ni clases.

1. Datos iniciales del negocio

El programa debe partir de una serie de datos económicos fijos.

Ingredientes mínimos

```
ingredientes = {  
    "harina": 1.20,  
    "tomate": 1.50,  
    "queso": 3.80,  
    "mozzarella": 4.20,  
    "jamon": 5.50,  
    "bacon": 6.00,  
    "pollo": 5.80,  
    "atun": 7.00,  
    "champinones": 3.20,  
    "cebolla": 1.10,  
    "aceitunas": 2.50,  
    "pepperoni": 6.50  
}
```

Puedes añadir más ingredientes si los necesitas para tus pizzas.

Gastos fijos y variables

```
gasto_suministros_por_pizza = 0.60  
gasto_personal_mensual = 4200  
gasto_alquiler_mensual = 1600  
iva = 0.10
```

El gasto de suministros representa luz, gas, agua, uso del horno, embalaje y pequeños gastos asociados a cada pizza.

2. Temporada y previsión de ventas

El programa deberá calcular una estimación mensual de pizzas vendidas según la temporada. La temporada puede funcionar como una especie de switch mediante match-case o mediante if, elif y else.

Temporada	Pizzas estimadas al mes
Baja	600 pizzas
Media	1000 pizzas
Alta	1500 pizzas

```
temporada = "media"

match temporada:
case "baja":
pizzas_estimadas_mes = 600
case "media":
pizzas_estimadas_mes = 1000
case "alta":
pizzas_estimadas_mes = 1500
case _:
pizzas_estimadas_mes = 1000
```

3. Lista de pizzas

El programa debe incluir una lista completa de al menos 20 pizzas. Cada pizza debe tener un nombre y una lista de ingredientes. Se recomienda usar un diccionario.

```
pizzas = {
"Margarita": ["harina", "tomate", "queso"],
"Prosciutto": ["harina", "tomate", "queso", "jamon"],
"Barbacoa": ["harina", "tomate", "queso", "bacon", "pollo"]
}
```

Cada alumno deberá ampliar esta estructura hasta llegar, como mínimo, a 20 pizzas diferentes.

4. Cálculo del precio de una pizza

El precio de una pizza no debe inventarse directamente. Debe calcularse a partir de varios elementos:

- Coste de los ingredientes utilizados.
- Gasto de suministros por pizza.
- Parte proporcional de gastos fijos mensuales.
- Margen de beneficio.
- IVA desglosado.

Fórmula orientativa

```
gastos_fijos_mensuales = gasto_personal_mensual + gasto_alquiler_mensual
coste_fijo_por_pizza = gastos_fijos_mensuales / pizzas_estimadas_mes

coste_total_pizza = coste_ingredientes + gasto_suministros_por_pizza +
coste_fijo_por_pizza
precio_sin_iva = coste_total_pizza * 1.35
precio_con_iva = precio_sin_iva * (1 + iva)
```

El margen de beneficio puede ser modificado por el alumno, pero deberá explicarse en el README.

5. Bebidas

El programa debe incluir bebidas. El cálculo de su precio de venta queda en manos del alumno, pero debe explicarse el criterio utilizado.

```
bebidas = {  
  "agua": 1.20,  
  "cola": 2.00,  
  "limonada": 2.00,  
  "cerveza": 2.50  
}
```

Ejemplos de criterios válidos: precio fijo, precio de compra más margen, precio por tamaño o bebidas incluidas en menús.

6. Carrito de compra

El programa debe permitir crear un carrito de compra. El carrito podrá contener pizzas, bebidas, menús u ofertas.

```
carrito = []  
  
carrito.append("Margarita")  
carrito.append("cola")
```

El programa debe mostrar:

- Productos comprados.
- Precio sin IVA.
- IVA desglosado.
- Precio final con IVA.

7. Ofertas

El programa debe incluir al menos 3 ofertas. Cada alumno decidirá cuáles aplica.

- Menú pizza + bebida.
- 2x1 en pizzas concretas.
- Descuento del 10% si el pedido supera 30 euros.
- Oferta de temporada.
- Oferta de día concreto.
- Menú familiar.

Las ofertas deben aplicarse mediante condicionales y funciones.

8. Funciones obligatorias

El programa debe organizarse mediante funciones. Como mínimo debe haber funciones equivalentes a estas:

```
calcular_coste_ingredientes()  
calcular_precio_pizza()  
mostrar_pizzas()  
agregar_producto_carrito()  
calcular_total_carrito()  
aplicar_oferta()  
mostrar_ticket()
```

No es obligatorio que tengan exactamente estos nombres, pero sí deben existir funciones que cumplan esas tareas.

9. Ticket final

El programa debe mostrar un ticket final parecido a este:

```
PIZZERÍA PYTHON

Productos:
- Pizza Margarita: 8.50 €
- Cola: 2.00 €

Base imponible: 9.55 €
IVA: 0.95 €
Total: 10.50 €

Gracias por su compra.
```

El IVA debe aparecer separado del total final.

10. Git y GitHub

El proyecto debe subirse a GitHub. El repositorio deberá incluir:

- Archivo principal .py.
- Archivo README.md.
- Al menos 5 commits.
- Mensajes de commit claros.

Gestión del proyecto con Jira o Trello

Si el proyecto se realiza en pareja o grupo, deberá existir una distribución clara de tareas usando Jira o Trello. No basta con repartirse el trabajo de palabra: las tareas deben estar creadas, asignadas y actualizadas en la herramienta elegida.

- Deberán crear un tablero en Jira o Trello para organizar el proyecto.
- Cada tarea deberá tener un responsable, una descripción y un estado.
- El tablero deberá reflejar el avance real del trabajo: tareas pendientes, en proceso, en revisión y terminadas.
- Las tareas deberán dividirse de forma razonable: pizzas, ingredientes, bebidas, carrito, ofertas, ticket, README, pruebas, etc.
- La memoria del proyecto deberá incluir capturas del tablero y explicar cómo se ha organizado el trabajo.

Ejemplos de commits

```
feat: crear estructura inicial del proyecto
feat: añadir listado de pizzas
feat: calcular precio de venta con IVA
feat: añadir carrito de compra
fix: corregir cálculo de ofertas
docs: completar README del proyecto
```

11. README obligatorio

El archivo README.md debe explicar:

- Qué hace el programa.

-
- Qué estructuras de datos se han usado.
 - Qué funciones tiene.
 - Qué ofertas se han incluido.
 - Cómo se calcula el precio de una pizza.
 - Cómo se calcula el IVA.
 - Qué criterio se ha usado para las bebidas.
 - Un ejemplo de ejecución del programa.

12. Memoria del proyecto

Además del código y del README, se deberá entregar una memoria del proyecto en formato PDF. Esta memoria documentará el proceso completo de trabajo, no solo el resultado final.

La memoria deberá incluir:

- Nombre de los integrantes del grupo.
- Distribución de tareas y responsabilidades.
- Capturas del tablero de Jira o Trello.
- Reuniones realizadas y acuerdos tomados.
- Decisiones importantes del proyecto.
- Errores encontrados durante el desarrollo y cómo se resolvieron.
- Cambios de criterio o modificaciones respecto a la idea inicial.
- Problemas de coordinación, si los ha habido, y cómo se han solucionado.
- Explicación del cálculo económico usado en el programa.
- Conclusión final sobre el trabajo realizado.

13. Restricciones

No se permite:

- Clases.
- Objetos propios.
- Librerías externas.
- Bases de datos.
- Interfaces gráficas.

Sí se permite:

- Variables.
- Listas, tuplas, diccionarios y conjuntos.
- Operadores, condicionales, bucles y funciones.
- Git y GitHub.

El proyecto debe poder entenderse y explicarse con los contenidos trabajados en clase.

14. Entrega

-
- Enlace al repositorio de GitHub.
 - Archivo .py funcionando.
 - Archivo README.md.
 - Memoria del proyecto en PDF.
 - Captura del historial de commits.
 - Capturas del tablero de Jira o Trello.
 - Breve explicación del cálculo de precios.

15. Criterios de evaluación

Criterio	Descripción
Funciones	El programa está organizado en funciones claras y reutilizables.
Estructuras de datos	Usa correctamente listas, diccionarios, tuplas o conjuntos.
Cálculo económico	Calcula costes, gastos, beneficio e IVA de forma coherente.
Carrito y ticket	Permite añadir productos y muestra un ticket claro.
Ofertas	Incluye al menos tres ofertas aplicadas correctamente.
GitHub	El repositorio está ordenado y tiene commits claros.
Gestión de tareas	El grupo usa correctamente Jira o Trello para organizar el trabajo.
Memoria	Documenta reuniones, decisiones, errores y cambios realizados durante el proyecto.
README	Explica el funcionamiento y las decisiones tomadas.